

Informatics 141 Computer Science 121

Information Retrieval

Lecture 7

Duplication of course material for any commercial purpose without the explicit written permission of the professor is prohibited.

These course materials borrow, with permission, from those of Prof. Cristina Videira Lopes, Prof. Alberto- Krone-Martins, Addison Wesley 2008, Chris Manning, Pandu Nayak, Hinrich Schütze, Heike Adel, Sascha Rothe, Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie. Powerpoint theme by Prof. André van der Hoek.

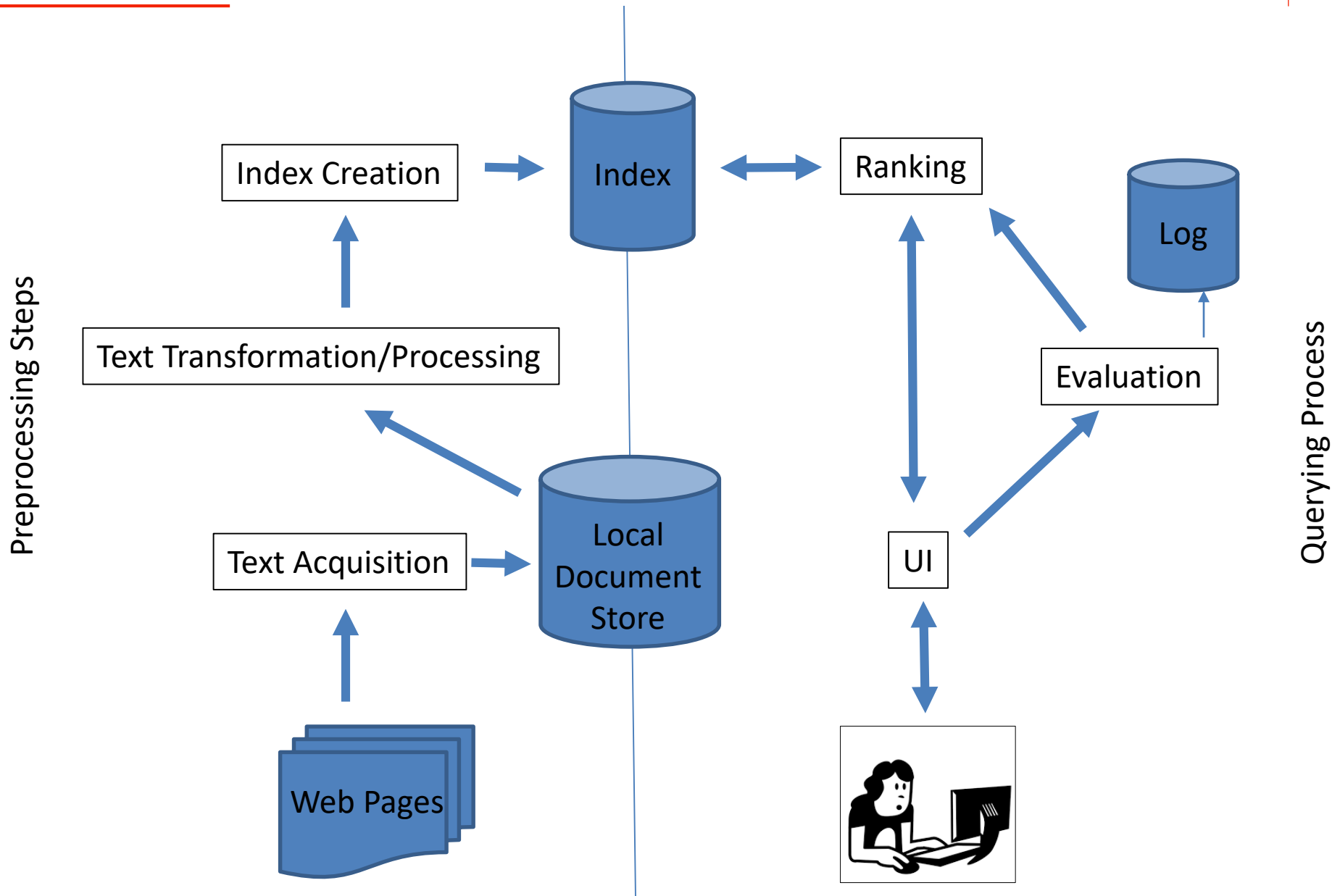
Ways of Acquiring Web Data

- Data dumps
 - Web APIs
 - Targeted downloads
-
- Web crawling ← last option

Crawling

Information Retrieval

Architecture



Basic Crawl Algorithm

- Initialize a queue of URLs (seeds)

Basic Crawl Algorithm

- Initialize a queue of URLs (seeds)
- Repeat until no more URLs in queue:

Basic Crawl Algorithm

- Initialize a queue of URLs (seeds)
- Repeat until no more URLs in queue:
 - Get one URL from the queue

Basic Crawl Algorithm

- Initialize a queue of URLs (seeds)
- Repeat until no more URLs in queue:
 - Get one URL from the queue
 - If the page can be crawled, fetch associated page

Basic Crawl Algorithm

- Initialize a queue of URLs (seeds)
- Repeat until no more URLs in queue:
 - Get one URL from the queue
 - If the page can be crawled, fetch associated page
 - Store representation of page

Basic Crawl Algorithm

- Initialize a queue of URLs (seeds)
- Repeat until no more URLs in queue:
 - Get one URL from the queue
 - If the page can be crawled, fetch associated page
 - Store representation of page
 - Extract URLs from page and add them to the queue
- Queue = “frontier”

Basic Crawl Algorithm

Crawled wepages

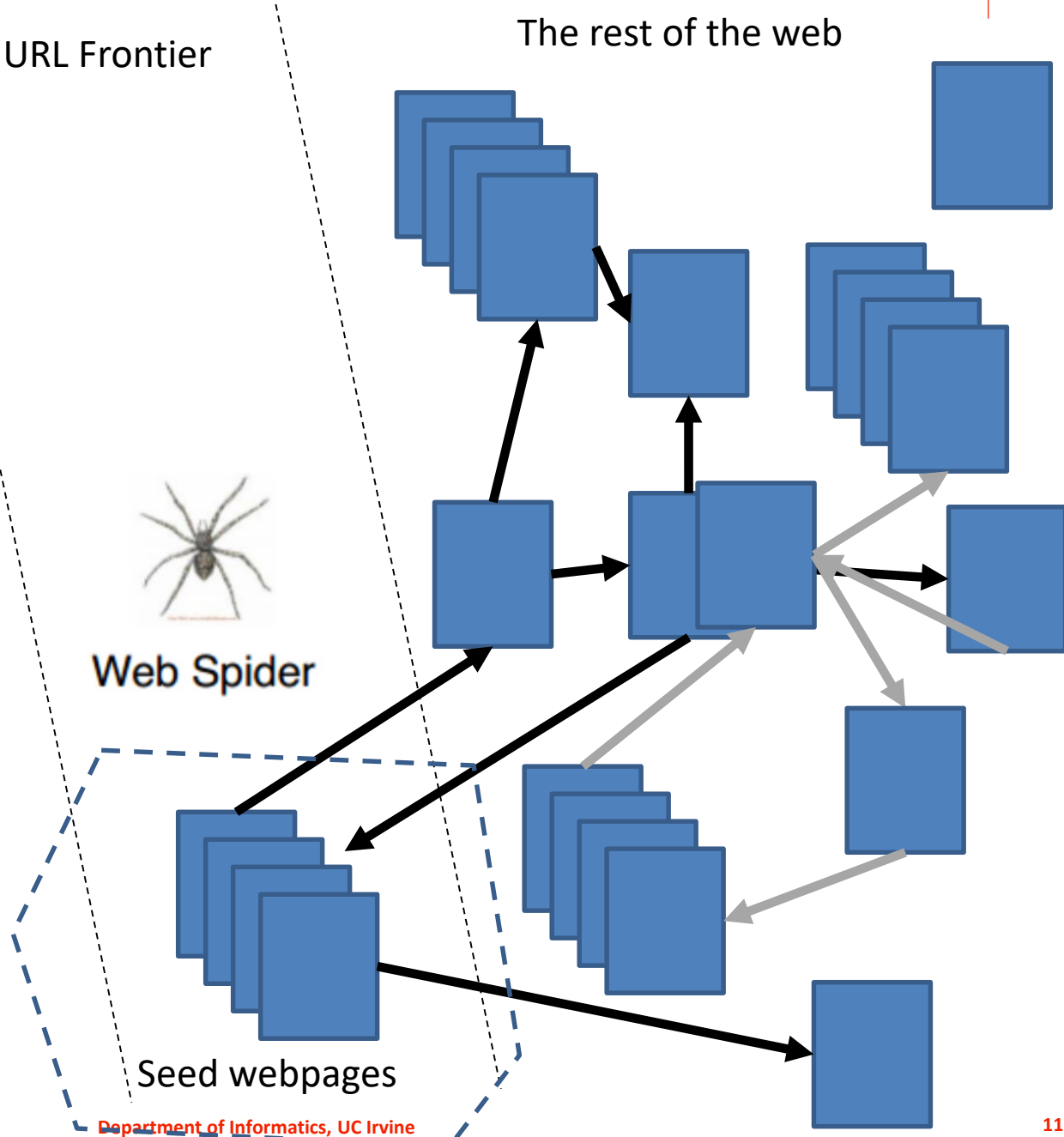
URL Frontier

The rest of the web



Web Spider

Seed webpages

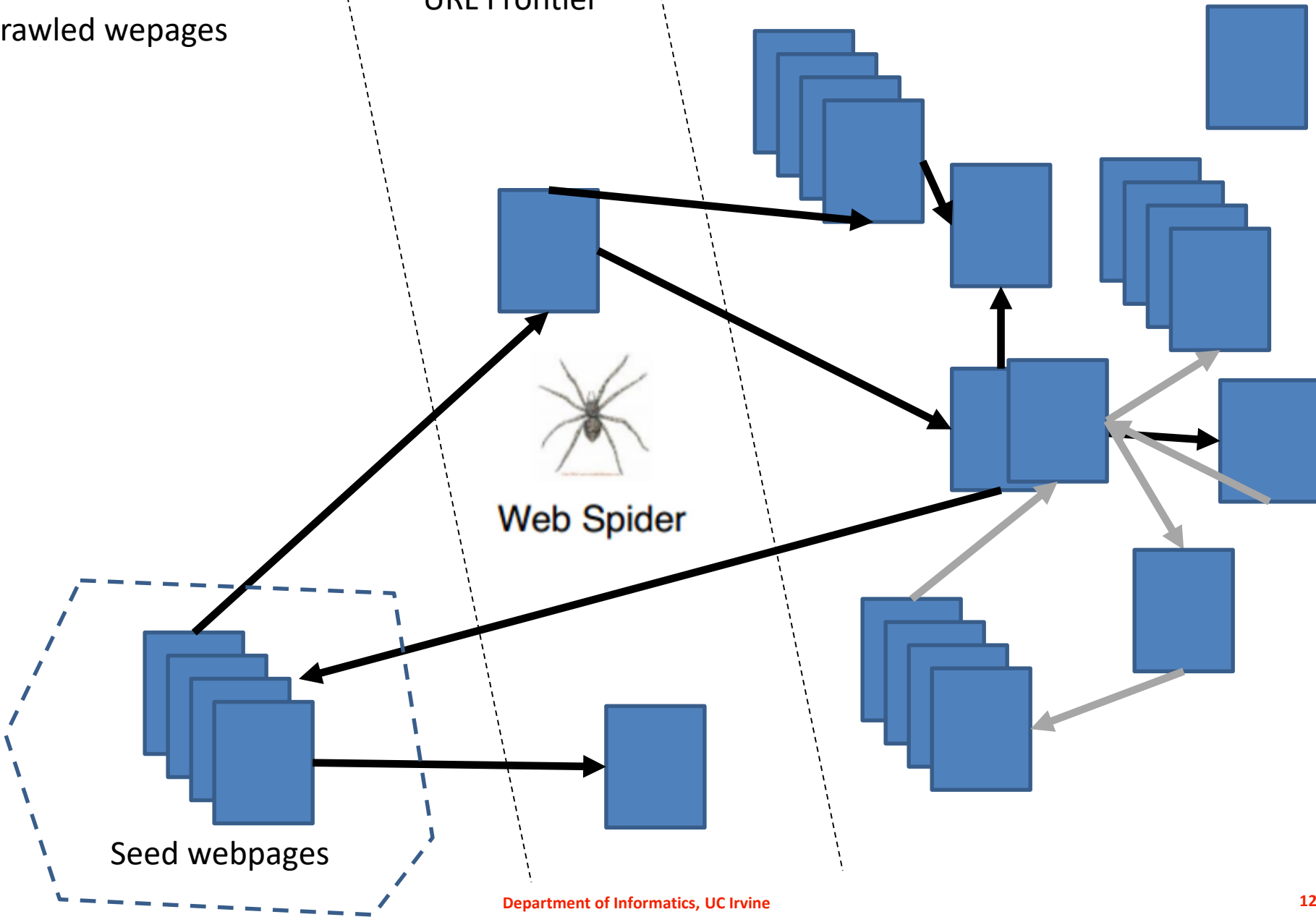


Basic Crawl Algorithm

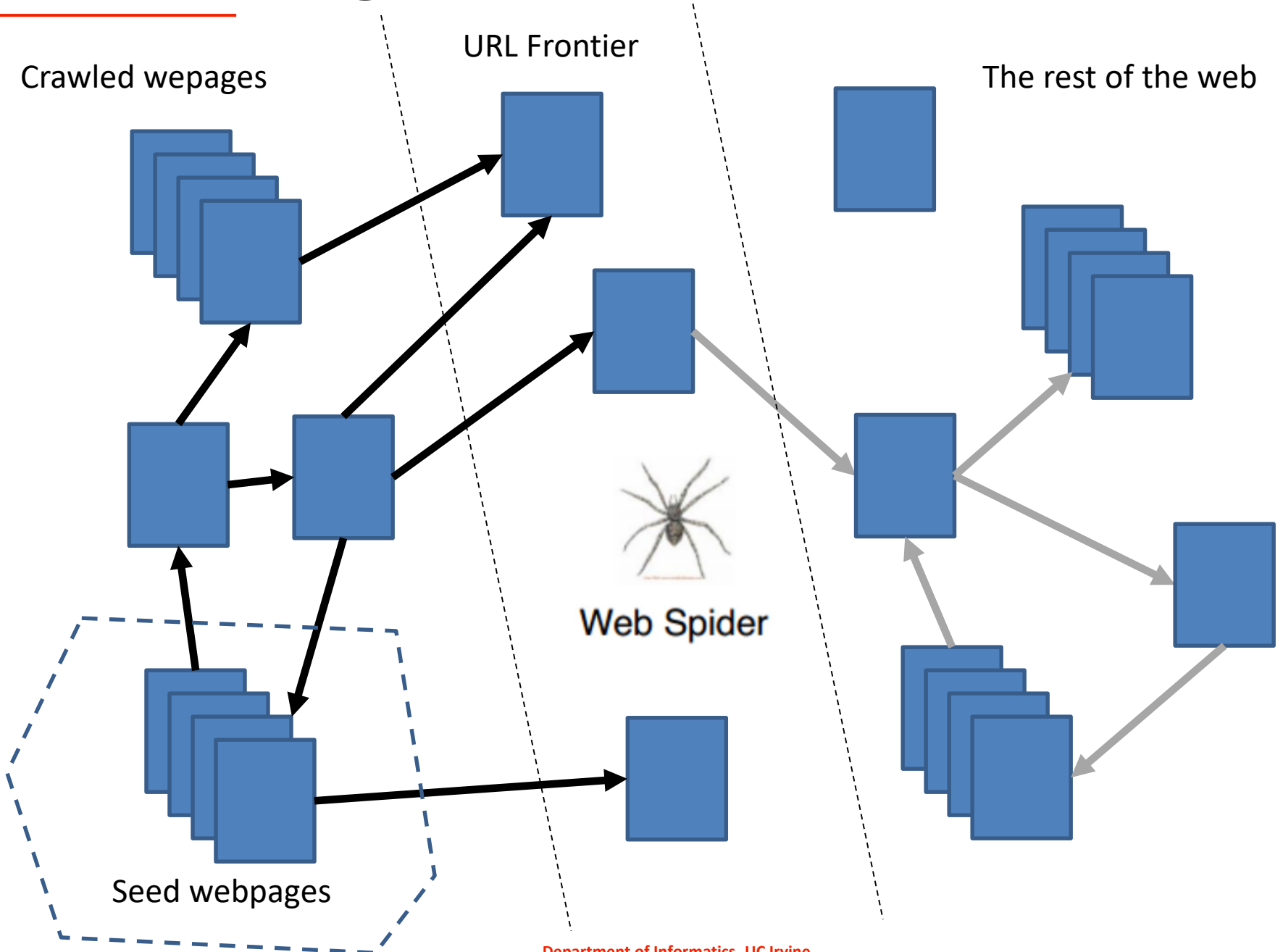
Crawled webpages

URL Frontier

The rest of the web



Basic Crawl Algorithm



Pseudo Code

```
procedure CRAWLERTHREAD(frontier)
  while not frontier.done() do
    website  $\leftarrow$  frontier.nextSite()
    url  $\leftarrow$  website.nextURL()
    if website.permitsCrawl(url) then
      text  $\leftarrow$  retrieveURL(url)
      storeDocument(url, text)
      for each url in parse(text) do
        frontier.addURL(url)
      end for
    end if
    frontier.releaseSite(website)
  end while
end procedure
```

Pseudo Code

```
procedure CRAWLER_THREAD(frontier)
  while not frontier.done() do
    website ← frontier.nextSite()
    url ← website.nextURL()
    if website.permitsCrawl(url) then
      text ← retrieveURL(url)
      storeDocument(url, text)
      for each url in parse(text) do
        frontier.addURL(url)
      end for
    end if
    frontier.releaseSite(website)
  end while
end procedure
```

Pseudo Code

```
procedure CRAWLER_THREAD(frontier)
  while not frontier.done() do
    website ← frontier.nextSite()
    url ← website.nextURL()
    if website.permitsCrawl(url) then
      text ← retrieveURL(url)
      storeDocument(url, text)
      for each url in parse(text) do
        frontier.addURL(url)
      end for
    end if
    frontier.releaseSite(website)
  end while
end procedure
```

But how do you know if the website allows you to crawl?

Permission to crawl



Permission to crawl

- Be polite: try to ask the website if you can crawl it first!
- Robots Exclusion Standard aka **robots.txt**
 - Sites may have that file at the root. Examples:
 - <http://www.cnn.com/robots.txt>
 - <http://en.wikipedia.org/robots.txt>
 - Very simple syntax (but no formal/official standard yet!):
 - <http://www.robotstxt.org/robotstxt.html>

Permission to crawl

Exclude all

```
User-agent: *  
Disallow: /
```

Allow all

```
User-agent: *  
Disallow:
```

Exclude all
from a part
of the site

```
User-agent: *  
Disallow: /cgi-bin/  
Disallow: /tmp/  
Disallow: /junk/
```

Exclude a
single robot

```
User-agent: BadBot  
Disallow: /
```

Allow a
single robot

```
User-agent: Google  
Disallow:  
  
User-agent: *  
Disallow: /
```

Permission to crawl

Exclude all

```
User-agent: *  
Disallow: /
```

Allow all

```
User-agent: *  
Disallow:
```

There is a precedence rule:

**IF THERE IS A SPECIFIC PART FOR A NAMED BOT,
THE RULE THAT IS VALID FOR THAT BOT,
IS THE RULE THAT HAS THE NAME OF THE BOT!**

Exclude a
single robot

```
User-agent: BadBot  
Disallow: /
```

Allow a
single robot

```
User-agent: Google  
Disallow:  
  
User-agent: *  
Disallow: /
```

Permission to crawl

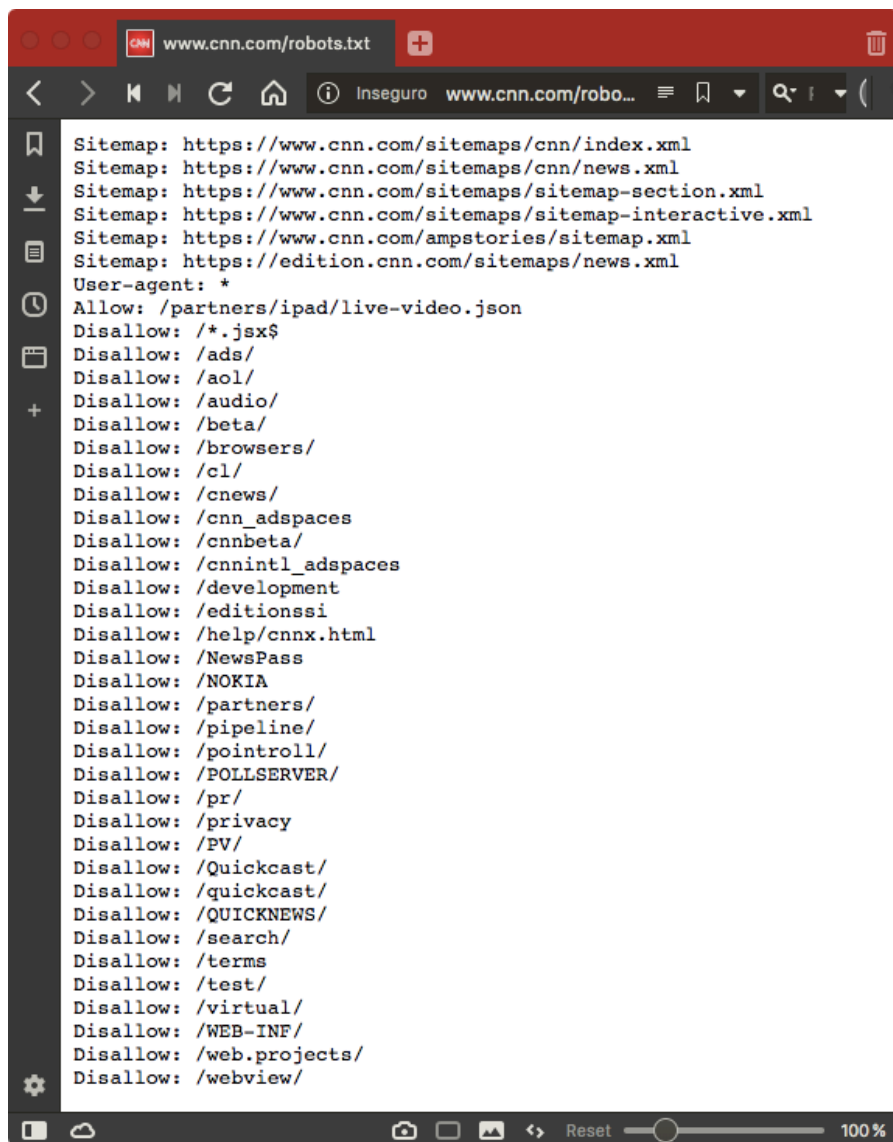
- Robots Exclusion Standard aka **robots.txt**
 - Sites may have that file at the root. Examples:
 - <http://www.cnn.com/robots.txt>
 - <http://en.wikipedia.org/robots.txt>
 - Very simple syntax (but no formal/official standard yet!):
 - <http://www.robotstxt.org/robotstxt.html>
 - **Honor basis!**
 - **It's not a security mechanism**



- Sitemaps (introduced by Google)
 - Also listed in robots.txt
 - Allow web masters to send info to crawlers. E.g.
 - Location of pages that might not be linked
 - Relative importance
 - Update frequency
- Example:
 - <http://www.cnn.com/robots.txt>

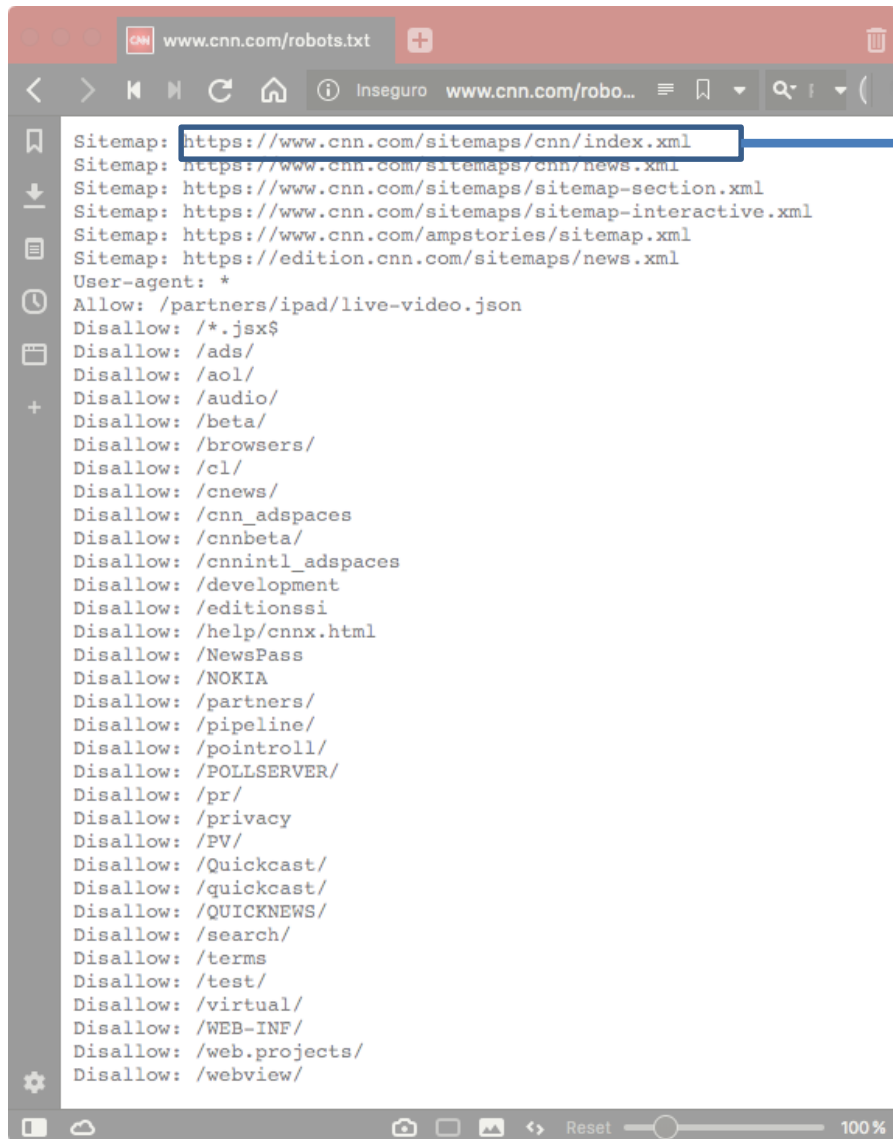
Information to crawlers

<http://www.cnn.com/robots.txt>



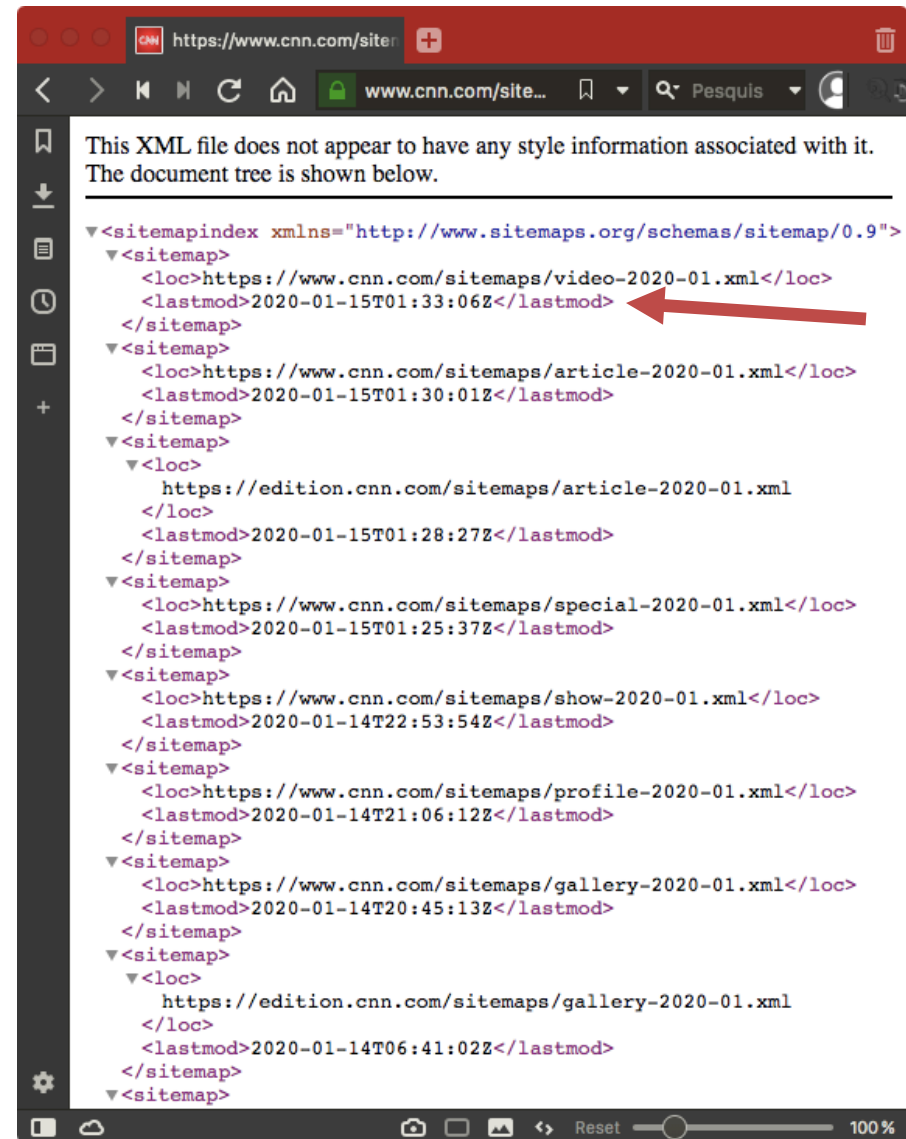
Information to crawlers

<http://www.cnn.com/robots.txt>



```
Sitemap: https://www.cnn.com/sitemaps/cnn/index.xml
Sitemap: https://www.cnn.com/sitemaps/cnn/news.xml
Sitemap: https://www.cnn.com/sitemaps/sitemap-section.xml
Sitemap: https://www.cnn.com/sitemaps/sitemap-interactive.xml
Sitemap: https://www.cnn.com/ampstories/sitemap.xml
Sitemap: https://edition.cnn.com/sitemaps/news.xml
User-agent: *
Allow: /partners/ipad/live-video.json
Disallow: /*.jsx$
Disallow: /ads/
Disallow: /aol/
Disallow: /audio/
Disallow: /beta/
Disallow: /browsers/
Disallow: /cl/
Disallow: /cnews/
Disallow: /cnn_adspaces
Disallow: /cnnbeta/
Disallow: /cnnintl_adspaces
Disallow: /development
Disallow: /editionssi
Disallow: /help/cnnx.html
Disallow: /NewsPass
Disallow: /NOKIA
Disallow: /partners/
Disallow: /pipeline/
Disallow: /pointroll/
Disallow: /POLLSERVER/
Disallow: /pr/
Disallow: /privacy
Disallow: /PV/
Disallow: /Quickcast/
Disallow: /quickcast/
Disallow: /QUICKNEWS/
Disallow: /search/
Disallow: /terms
Disallow: /test/
Disallow: /virtual/
Disallow: /WEB-INF/
Disallow: /web.projects/
Disallow: /webview/
```

<https://www.cnn.com/sitemaps/cnn/index.xml>



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" ?>
<sitemapindex xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <sitemap>
    <loc>https://www.cnn.com/sitemaps/video-2020-01.xml</loc>
    <lastmod>2020-01-15T01:33:06Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://www.cnn.com/sitemaps/article-2020-01.xml</loc>
    <lastmod>2020-01-15T01:30:01Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://edition.cnn.com/sitemaps/article-2020-01.xml</loc>
    <lastmod>2020-01-15T01:28:27Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://www.cnn.com/sitemaps/special-2020-01.xml</loc>
    <lastmod>2020-01-15T01:25:37Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://www.cnn.com/sitemaps/show-2020-01.xml</loc>
    <lastmod>2020-01-14T22:53:54Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://www.cnn.com/sitemaps/profile-2020-01.xml</loc>
    <lastmod>2020-01-14T21:06:12Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://www.cnn.com/sitemaps/gallery-2020-01.xml</loc>
    <lastmod>2020-01-14T20:45:13Z</lastmod>
  </sitemap>
  <sitemap>
    <loc>https://edition.cnn.com/sitemaps/gallery-2020-01.xml</loc>
    <lastmod>2020-01-14T06:41:02Z</lastmod>
  </sitemap>
</sitemapindex>
```

Information to crawlers

<http://www.cnn.com/robots.txt>

<https://www.cnn.com/sitemaps/cnn/index.xml>

The image displays two browser windows side-by-side. The left window shows the robots.txt file for CNN, listing various disallowed paths. The right window shows an XML sitemap for CNN, which lists URLs and their metadata. A red arrow points to the `<changefreq>daily</changefreq>` tag in the XML, indicating the frequency at which the search engines should crawl the page.

Left Window: robots.txt

```
Sitemap: https://www.cnn.com/sitemaps/
Sitemap: https://www.cnn.com/sitemaps/
Sitemap: https://www.cnn.com/sitemaps/
Sitemap: https://www.cnn.com/sitemaps/
Sitemap: https://www.cnn.com/ampstorid
Sitemap: https://edition.cnn.com/site
User-agent: *
Allow: /partners/ipad/live-video.json
Disallow: /*.jsx$
Disallow: /ads/
Disallow: /aol/
Disallow: /audio/
Disallow: /beta/
Disallow: /browsers/
Disallow: /cl/
Disallow: /cnews/
Disallow: /cnn_adspaces
Disallow: /cnnbeta/
Disallow: /cnnintl_adspaces
Disallow: /development
Disallow: /editionssi
Disallow: /help/cnnx.html
Disallow: /NewsPass
Disallow: /NOKIA
Disallow: /partners/
Disallow: /pipeline/
Disallow: /pointroll/
Disallow: /POLLSERVER/
Disallow: /pr/
Disallow: /privacy
Disallow: /PV/
Disallow: /Quickcast/
Disallow: /quickcast/
Disallow: /QUICKNEWS/
Disallow: /search/
Disallow: /terms
Disallow: /test/
Disallow: /virtual/
Disallow: /WEB-INF/
Disallow: /web.projects/
Disallow: /webview/
```

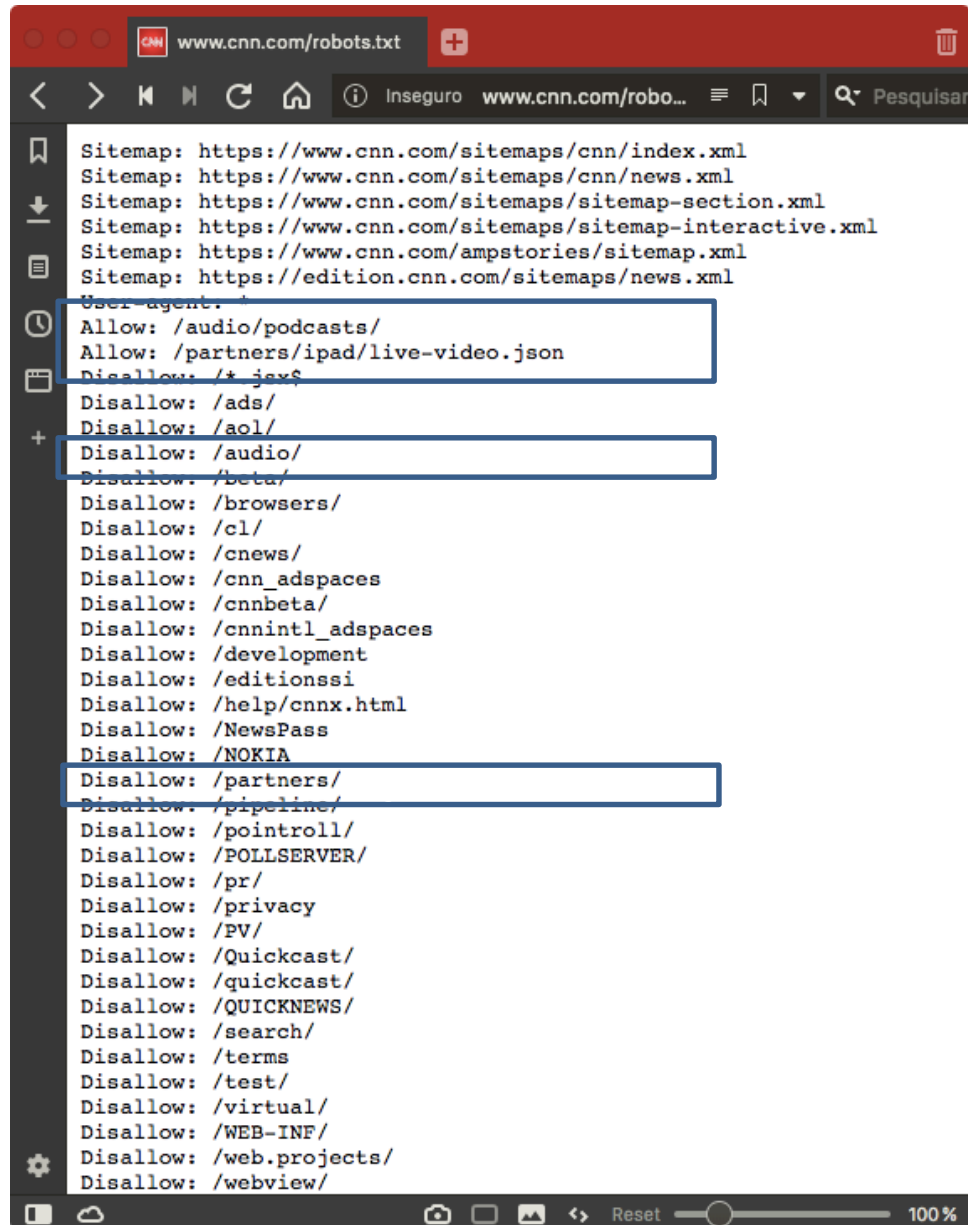
Right Window: XML Sitemap

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" ?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
  xmlns:xhtml="http://www.w3.org/1999/xhtml"
  xmlns:image="http://www.google.com/schemas/sitemap-image/1.1">
  <url>
    <loc>
      https://www.cnn.com/2012/01/25/world/gallery/wus-us-
      mia/index.html
    </loc>
    <lastmod>2012-01-25T18:24:37Z</lastmod>
    <changefreq>daily</changefreq>
    <priority>0.5</priority>
    <image:image>
      <image:loc>
        https://i2.cdn.turner.com/cnnnext/dam/assets/120111071718-
        jpac1-plane-story-top.jpg
      </image:loc>
      <image:caption>
        Gunner Sgt. Bryon Bebout with the wreckage of a B-24
        Liberator during excavation operations in Papua New Guinea.
        The JPAC recovery team were searching for nine Americans
        that remain unaccounted-for from World War II.
      </image:caption>
    </image:image>
  </url>
  <url>
    <loc>
      https://www.cnn.com/2012/01/18/living/gallery/cindy-
      costa/index.html
    </loc>
    <lastmod>2012-01-18T17:21:49Z</lastmod>
    <changefreq>daily</changefreq>
    <priority>0.5</priority>
    <image:image>
      <image:loc>
        https://i2.cdn.turner.com/cnnnext/dam/assets/120117082827-
        cindy-costa-01-story-top.jpg
      </image:loc>
      <image:caption>
```

Information to crawlers

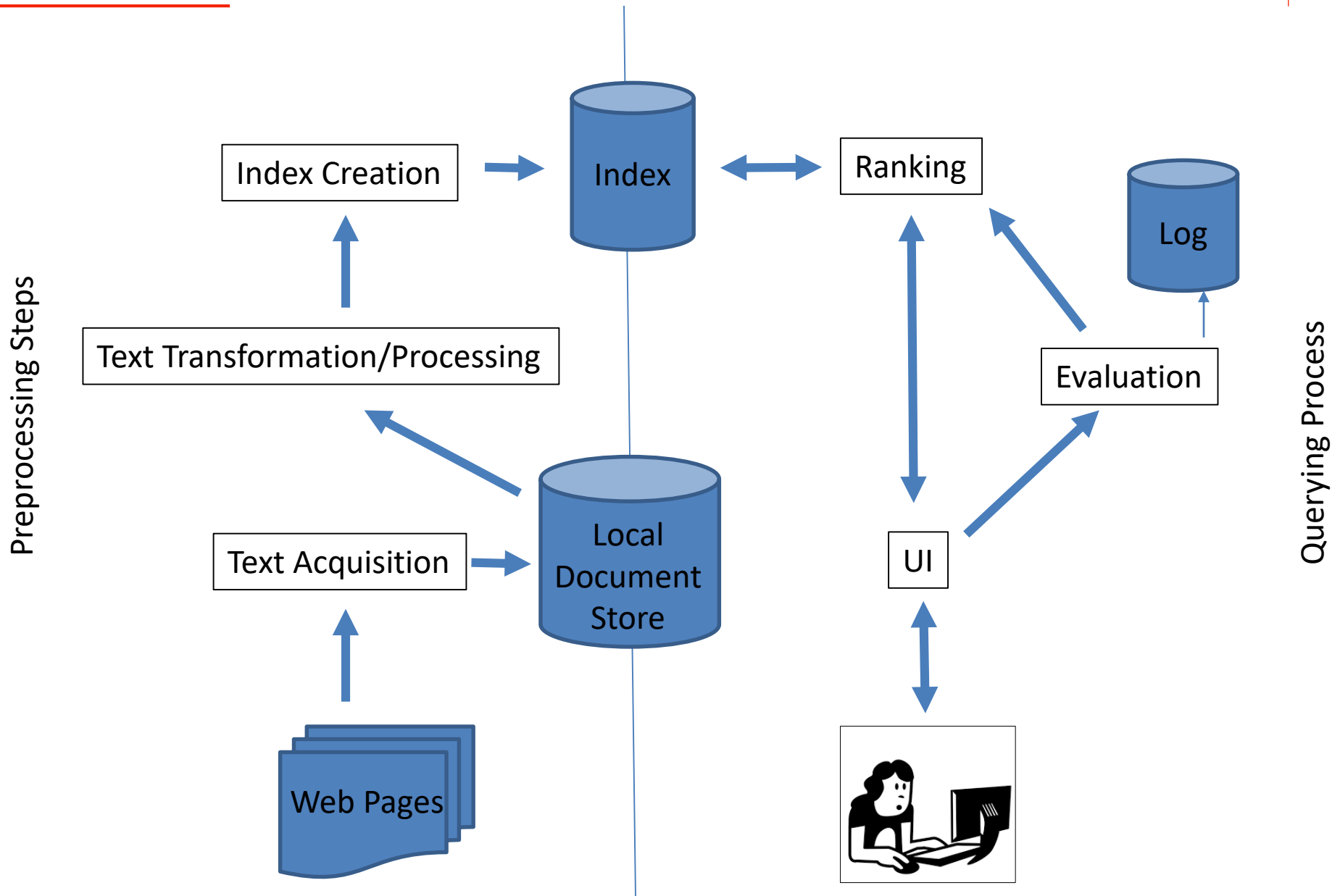
<http://www.cnn.com/robots.txt>



```
Sitemap: https://www.cnn.com/sitemaps/cnn/index.xml
Sitemap: https://www.cnn.com/sitemaps/cnn/news.xml
Sitemap: https://www.cnn.com/sitemaps/sitemap-section.xml
Sitemap: https://www.cnn.com/sitemaps/sitemap-interactive.xml
Sitemap: https://www.cnn.com/ampstories/sitemap.xml
Sitemap: https://edition.cnn.com/sitemaps/news.xml
User-agent: *
Allow: /audio/podcasts/
Allow: /partners/ipad/live-video.json
Disallow: /*.js$
Disallow: /ads/
Disallow: /aol/
Disallow: /audio/
Disallow: /beta/
Disallow: /browsers/
Disallow: /cl/
Disallow: /cnews/
Disallow: /cnn_adspaces
Disallow: /cnnbeta/
Disallow: /cnnintl_adspaces
Disallow: /development
Disallow: /editionssi
Disallow: /help/cnnx.html
Disallow: /NewsPass
Disallow: /NOKIA
Disallow: /partners/
Disallow: /pipeline/
Disallow: /pointroll/
Disallow: /POLLSERVER/
Disallow: /pr/
Disallow: /privacy
Disallow: /PV/
Disallow: /Quickcast/
Disallow: /quickcast/
Disallow: /QUICKNEWS/
Disallow: /search/
Disallow: /terms
Disallow: /test/
Disallow: /virtual/
Disallow: /WEB-INF/
Disallow: /web.projects/
Disallow: /webview/
```

- You can allow a specific resource from a disallowed resource
 - For instance: *you can disallow to crawl some folder, but allow the bot to crawl a resource that contains a list of links that you want to be indexed.*

Architecture



Pseudo Code

```
procedure CRAWLER_THREAD(frontier)
  while not frontier.done() do
    website ← frontier.nextSite()
    url ← website.nextURL()
    if website.permitsCrawl(url) then
      text ← retrieveURL(url)
      storeDocument(url, text)
      for each url in parse(text) do
        frontier.addURL(url)
      end for
    end if
    frontier.releaseSite(website)
  end while
end procedure
```

“Basic algorithm” is...

- Theoretically correct

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**
 1. Will upset web admins (impolite)
 - It's abusing the web servers

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**
 1. Will upset web admins (impolite)
 - It's abusing the web servers
 2. Very slow
 - 1 page at a time

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**
 1. Will upset web admins (impolite)
 - It's abusing the web servers
 2. Very slow
 - 1 page at a time
 3. Will get caught in **traps and infinite sequences**

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**
 1. Will upset web admins (impolite)
 - It's abusing the web servers
 2. Very slow
 - 1 page at a time
 3. Will get caught in traps and infinite sequences
 4. Will fetch duplicates without noticing

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**
 1. Will upset web admins (impolite)
 - It's abusing the web servers
 2. Very slow
 - 1 page at a time
 3. Will get caught in traps and infinite sequences
 4. Will fetch duplicates without noticing
 5. Will bring in data noise

“Basic algorithm” is...

- Theoretically correct
- **Seriously lacking to use in practice**
 1. Will upset web admins (impolite)
 - It's abusing the web servers
 2. Very slow
 - 1 page at a time
 3. Will get caught in traps and infinite sequences
 4. Will fetch duplicates without noticing
 5. Will bring in data noise
 6. Will miss content due to client-side scripting

1. Politeness

- Avoid hitting any site too often
 - Sites are for people, not for bots
- Ignore politeness → Denial of service (DOS) attack
- **Be polite** → Use artificial delays

2. Performance (I)

- Back of the envelope calculation:
 - 1 page fetch = 500ms
 - How much time to crawl 1 million pages?
 - (it's worse than that... Unresponsive servers)
- Most of the time, the crawler thread is waiting for the network data
- Solution: multi-threaded or distributed crawling
 - Politeness is harder to control, but it is possible (e.g. different servers)

2. Performance (II)

- Domain Name lookups
 - Given a domain name, retrieve its IP address
 - www.ics.uci.edu -> 128.195.1.83
- Distributed set of servers
 - Latency can be high (2 secs is not unusual)
- Common implementations are blocking
 - One request at a time
 - Result is cached
- Back of the envelope calculation:
 - 1 DNS lookup → 800ms
 - How much time to lookup the entire Web?

3. Crawler traps

- **May** trap the crawler on the site forever
 - Web server responds with ever changing URLs and content
 - Dynamic pages
 - Can be intentional or unintentional
 - E.g. the ICS calendar is a crawler trap
 - Some webadmins can create traps to penalize impolite crawlers
- See <http://www.fleiner.com/bots/>
 - E.g. very large documents, disallowed in robots.txt, created to consume crawler resources or event to break poorly designed parsers of crawlers that ignore robots.txt

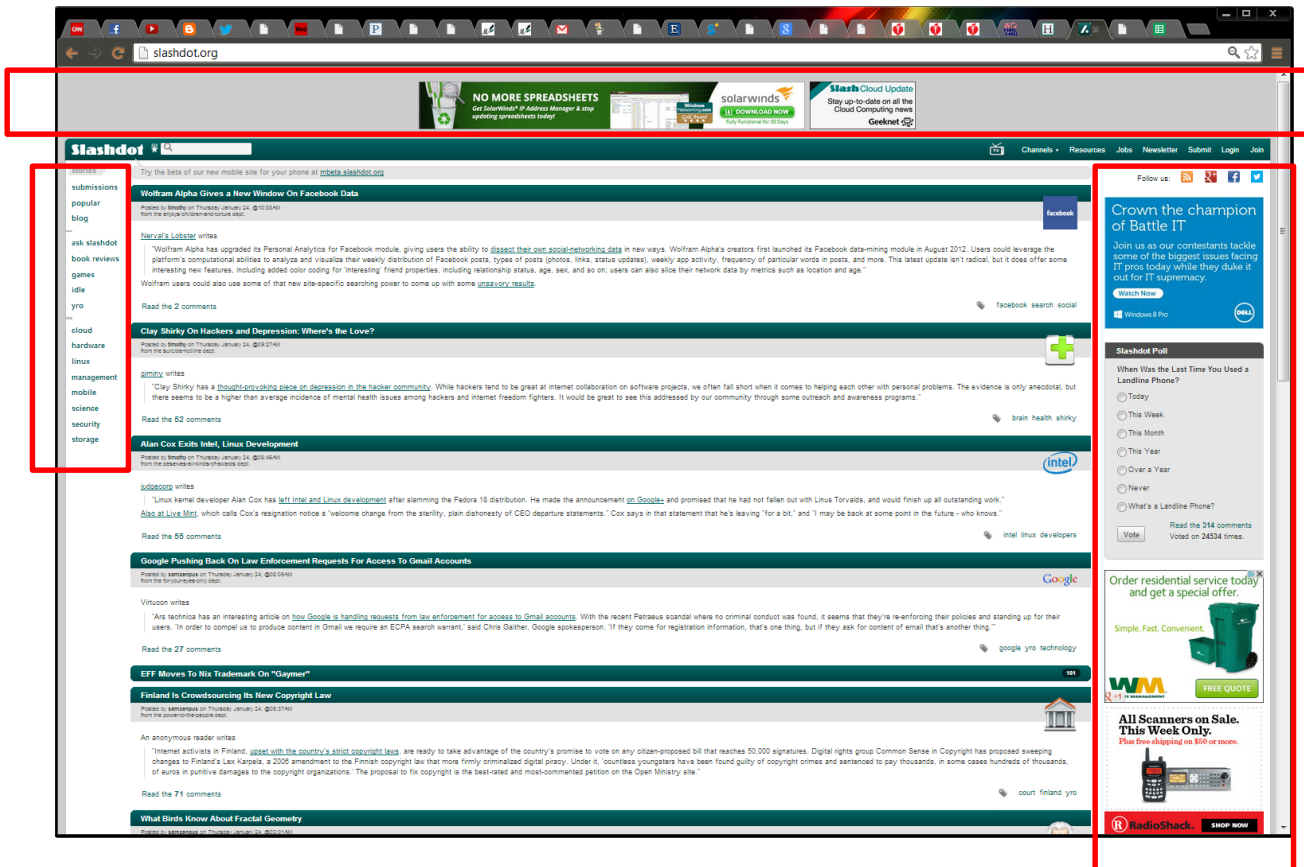


4. Duplicate Detection

- Exact and near duplication are widespread
 - Copies, mirror sites, versions, spam, plagiarism...
 - Studies: 30% of Web pages are [near-]duplicates of the other 70%
 - Little or no value (noise to the search engine and the user; you can show only one to the user and perhaps a “show similar” link)
- Detection
 - Detection of exact duplication is easy, but *exact duplication is rare*
 - Hashes, checksums
 - Detection of *near-duplicates* is harder
 - Page fingerprints

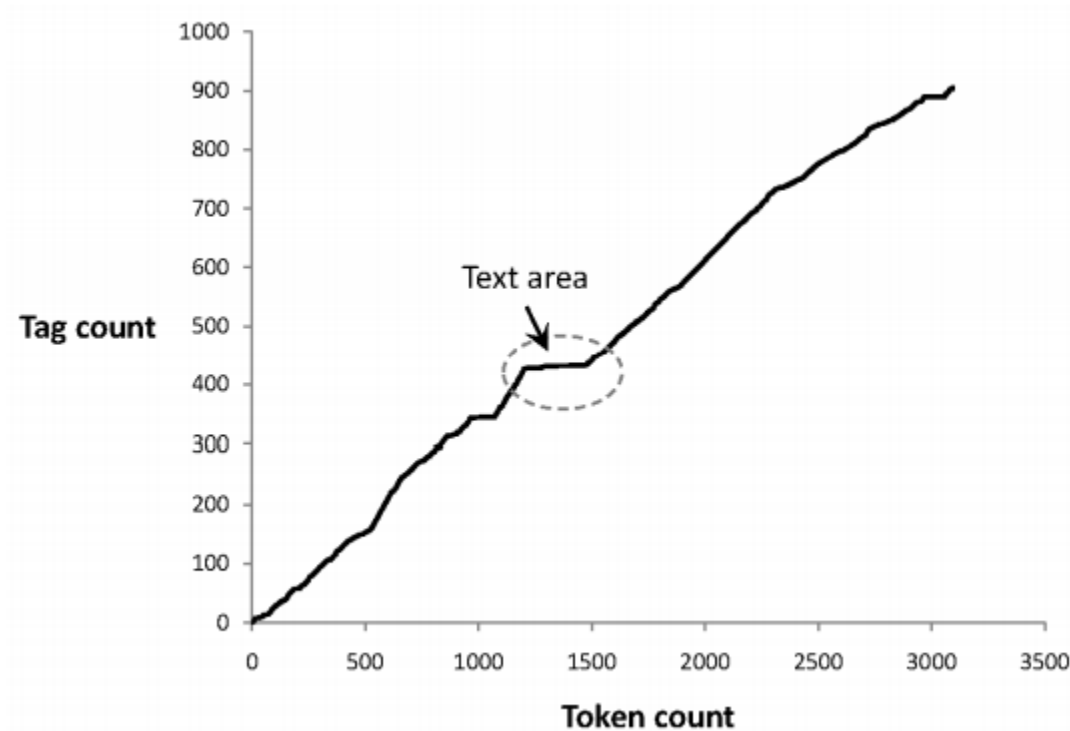
5. Data Noise

- Web pages have content not directly related to the page
 - Ads, templates, etc
 - Noise negatively impacts information retrieval



5. Data Noise : Finding Content Blocks

- Technique 1: Cumulative distribution of tags
 - Document slope curve (e.g. Finn, Kushmerick & Smyth, 2001)



- Other techniques in literature

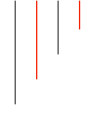
6. Client-Side Scripting

- Modern web sites are heavily scripted (JavaScript, TypeScript)
 - Content behind XMLHttpRequests

6. Client-Side Scripting

- Modern web sites are heavily scripted (JavaScript, TypeScript)
 - Content behind XMLHttpRequests
- To get to that content crawlers must interact with the scripts
 - Hard thing to do: user interaction emulation (e.g. Selenium)
 - Most crawlers will not do it and the content will be never indexed

The Deep Web



- Places where crawlers rarely go...
 - Content behind login forms
 - Content behind JavaScript/TypeScript
 - Sites that aren't linked from anywhere
- It is estimated that the deep web is 400-500x larger than the shallow web [<http://dx.doi.org/10.3998/3336451.0007.104>]